Report: Fake News Detection

Team Member **: Shashank Lokesha**

Objective

The objective of this assignment is to develop a Semantic Classification model. You will be using Word2Vec method to extract the semantic relations from the text and develop a basic understanding of how to train supervised models to categorise text based on its meaning, rather than just syntax. You will explore how this technique is used in situations where understanding textual meaning plays a critical role in making accurate and efficient decisions.

Business Objective

The spread of fake news has become a significant challenge in today's digital world. With the massive volume of news articles published daily, it's becoming harder to distinguish between credible and misleading information. This creates a need for systems that can automatically classify news articles as true or fake, helping to reduce misinformation and protect public trust.

In this assignment, you will develop a Semantic Classification model that uses the Word2Vec method to detect recurring patterns and themes in news articles. Using supervised learning models, the goal is to build a system that classifies news articles as either fake or true.

Pipelines that needs to be performed

You need to perform the following tasks to complete the assignment:

1. Data Preparation
2. Text Preprocessing
3. Train Validation Split
4. EDA on Training Data
5. EDA on Validation Data [Optional]
6. Feature Extraction
7. Model Training and Evaluation

Data Dictionary

For this assignment, you will work with two datasets, `True.csv` and `Fake.csv`. Both datasets contain three columns:

- title of the news article
- text of the news article
- date of article publication

`True.csv` dataset includes 21,417 true news, while the `Fake.csv` dataset comprises 23,502 fake news.

Installing required Libraries ¶

[1]:
```
!pip install --upgrade numpy==1.26.4
!pip install --upgrade pandas==2.2.2
!pip install --upgrade nltk==3.9.1
!pip install --upgrade spacy==3.7.5
!pip install --upgrade scipy==1.12
!pip install --upgrade pydantic==2.10.5
!pip install wordcloud==1.9.4
!python -m spacy download en_core_web_sm
```

Importing the necessary libraries

```
# Import essential libraries for data manipulation and analysis
import numpy as np  # For numerical operations and arrays
import pandas as pd  # For working with dataframes and structured data
import re  # For regular expression operations (text processing)
import nltk  # Natural Language Toolkit for text processing
import spacy  # For advanced NLP tasks
import string  # For handling string-related operations

# Optional: Uncomment the line below to enable GPU support for spaCy (if you have a compatible GPU)
#spacy.require_gpu()

# Load the spaCy small English language model
nlp = spacy.load("en_core_web_sm")

# For data visualization
import seaborn as sns  # Data visualization library for statistical graphics
import matplotlib.pyplot as plt  # Matplotlib for creating static plots
# Configure Matplotlib to display plots inline in Jupyter Notebook
%matplotlib inline

# Suppress unnecessary warnings to keep output clean
import warnings
warnings.filterwarnings('ignore')

# For interactive plots
from plotly.offline import plot  # Enables offline plotting with Plotly
import plotly.graph_objects as go  # For creating customizable Plotly plots
import plotly.express as px  # A high-level interface for Plotly

# For preprocessing and feature extraction in machine learning
from sklearn.feature_extraction.text import (  # Methods for text vectorization
    CountVectorizer,  # Converts text into a bag-of-words model
)

# Import accuracy, precision, recall, f_score from sklearn to predict train accuracy
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Pretty printing for better readability of output
from pprint import pprint

# For progress tracking in loops (useful for larger datasets)
from tqdm import tqdm, tqdm_notebook  # Progress bar for loops
tqdm.pandas()  # Enables progress bars for pandas operations
```

```
## Change the display properties of pandas to max
# Show full content of each column
pd.set_option('display.max_colwidth', None)

# Display all columns in a DataFrame
pd.set_option('display.max_columns', None)

# Display all rows in a DataFrame
pd.set_option('display.max_rows', None)

# Set display width to auto-detect
pd.set_option('display.width', None)

# Improve column alignment
pd.set_option('display.expand_frame_repr', False)
```

Load the data

Load the True.csv and Fake.csv files as DataFrames

```
# Import the first file - True.csv
true_df = pd.read_csv("True.csv")

# Import the second file - Fake.csv
fake_df = pd.read_csv("Fake.csv")
```

```
print("True News Dataset Shape:", true_df.shape)
print("Fake News Dataset Shape:", fake_df.shape)
```

```
True News Dataset Shape: (21417, 3)
Fake News Dataset Shape: (23523, 3)
```

1. Data Preparation [10 marks]

1.0 Data Understanding

```
# Inspect the DataFrame with True News to understand the given data
true_df.head()
```

	title	
0	As U.S. budget fight looms, Republicans flip their fiscal script	1.5trillion,tobepaidoffbyfuturegenerations,tofinancetaxcutsforcorporationsandtherich.\nThisisoneoftheleast...fiscallyresponsiblebillswe'veverse
1	U.S. military to accept transgender recruits on Monday: Pentagon	WASHINGTON (Reuters) - Transgender people will be allowed for the first time to enlist in the U.S. military starting on Monday as ordered by federal courts, the Pent
	Senior U.S. Republican	

```
# Inspect the DataFrame with Fake News to understand the given data
fake_df.head()
```

	title	text	date
0	Donald Trump Sends Out Embarrassing New Year's Eve Message; This is Disturbing	Donald Trump just couldn t wish all Americans a Happy New Year and leave it at that. Instead, he had to give a shout out to his enemies, haters and the very dishonest fake news media. The former reality show star had just one job to do and he couldn t do it. As our Country rapidly grows stronger and smarter, I want to wish all of my friends, supporters, enemies, haters, and even the very dishonest Fake News Media, a Happy and Healthy New Year, President Angry Pants tweeted. 2018 will be a great year for America! As our Country rapidly grows stronger and smarter, I want to wish all of my friends, supporters, enemies, haters, and even the very dishonest Fake News Media, a Happy and Healthy New Year. 2018 will be a great year for America! Donald J. Trump (@realDonaldTrump) December 31, 2017Trump s tweet went down about as welll as you d expect.What kind of president sends a New Year s greeting like this despicable, petty, infantile gibberish? Only Trump! His lack of decency won t even allow him to rise above the gutter long enough to wish the American citizens a happy new year! Bishop Talbert Swan (@TalbertSwan) December 31, 2017no one likes you Calvin (@calvinstowell) December 31, 2017Your impeachment would make 2018 a great year for America, but I ll also accept regaining control of Congress. Miranda Yaver (@mirandayaver) December 31, 2017Do you hear yourself talk? When you have to include that many people that hate you you have to wonder? Why do the they all hate me? Alan Sandoval (@AlanSandoval13) December 31, 2017Who uses the word Haters in a New Years wish?? Marlene (@marlene399) December 31, 2017You can t just say happy new year? Koren pollitt (@Korencarpenter) December 31, 2017Here s Trump s New Year s Eve tweet from 2016.Happy New Year to all, including to my many enemies and those who have fought me and lost so badly they just don t know what to do. Love! Donald J. Trump (@realDonaldTrump) December 31, 2016This is nothing new for Trump. He s been doing this for years.Trump has directed messages to his enemies and haters for New Year s, Easter, Thanksgiving, and the anniversary of 9/11. pic.twitter.com/4FPAe2KypA Daniel Dale (@ddale8) December 31, 2017Trump s holiday tweets are clearly not presidential.How long did he work at Hallmark before becoming President? Steven Goodine (@SGoodine) December 31, 2017He s always been like this . . . the only difference is that in the last few years, his filter has been breaking down. Roy Schulze (@thbthttt) December 31, 2017Who, apart from a teenager uses the term haters? Wendy (@WendyWhistles) December 31, 2017he s a fucking 5 year old Who Knows (@rainyday80) December 31, 2017So, to all the people who voted for this a hole thinking he would change once he got into power, you were wrong! 70-year-old men don t change and now he s a year older.Photo by Andrew Burton/Getty Images.	December 31, 2017
1	Drunk Bragging Trump Staffer Started Russian Collusion Investigation	House Intelligence Committee Chairman Devin Nunes is going to have a bad day. He s been under the assumption, like many of us, that the Christopher Steele-dossier was what prompted the Russia investigation so he s been lashing out at the Department of Justice and the FBI in order to protect Trump. As it happens, the dossier is not what started the investigation, according to documents obtained by the New York Times.Former Trump campaign adviser George Papadopoulos was drunk in a wine bar when he revealed knowledge of Russian opposition research on Hillary Clinton.On top of that, Papadopoulos wasn t just a covfefe boy for Trump, as his administration has alleged. He had a much larger role, but none so damning as being a drunken fool in a wine bar. Coffee boys don t help to arrange a New York meeting between Trump and President Abdel Fattah el-Sisi of Egypt two months before the election. It was known before that the former aide set up meetings with world leaders for Trump, but team Trump ran with him being merely a coffee boy.In May 2016, Papadopoulos revealed to Australian diplomat Alexander Downer that Russian officials were shopping around possible dirt on then-Democratic presidential nominee Hillary Clinton. Exactly how much Mr. Papadopoulos said that night at the Kensington Wine Rooms with the Australian, Alexander Downer, is unclear, the report states. But two months later, when leaked Democratic emails began appearing online, Australian officials passed the information about Mr. Papadopoulos to their American counterparts, according to four current and former American and foreign officials with direct knowledge of the Australians role. Papadopoulos pleaded guilty to lying to the F.B.I. and is now a cooperating witness with Special Counsel Robert Mueller s team.This isn t a presidency. It s a badly scripted reality TV show.Photo by Win McNamee/Getty Images.	December 31, 2017
		On Friday, it was revealed that former Milwaukee Sheriff David Clarke, who was being considered for Homeland Security Secretary in Donald Trump s administration, has an email scandal of his own.In January, there was a brief run-in on a plane between Clarke and fellow passenger Dan Black, who he later had detained by the police for no reason whatsoever, except that maybe his feelings were hurt. Clarke messaged the police to stop Black after he deplaned, and now, a search warrant has been executed by the FBI to see the exchanges.Clarke is calling it fake news even though copies of the search warrant are on the Internet. I am UNINTIMIDATED by lib media attempts to smear and discredit me with their FAKE NEWS reports designed to silence me, the former sheriff tweeted. I will continue to poke them in the eye with a sharp stick and bitch slap these scum bags til they get it. I have been attacked by better people than them #MAGA I am UNINTIMIDATED by lib media attempts to smear and discredit me with their FAKE NEWS reports designed to silence me. I will continue to poke them in the eye with a sharp stick and bitch slap these scum bags til they get it. I have been attacked by	

```python
# Print the column details for True News DataFrame
true_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21417 entries, 0 to 21416
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   title   21417 non-null  object
 1   text    21417 non-null  object
 2   date    21417 non-null  object
dtypes: object(3)
memory usage: 502.1+ KB
```

```python
# Print the column details for Fake News Dataframe
fake_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23523 entries, 0 to 23522
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   title   23502 non-null  object
 1   text    23502 non-null  object
 2   date    23481 non-null  object
dtypes: object(3)
memory usage: 551.4+ KB
```

```python
# Print the column names of both DataFrames
print("True News Columns:", list(true_df.columns))
print("Fake News Columns:", list(fake_df.columns))
```

```
True News Columns: ['title', 'text', 'date']
Fake News Columns: ['title', 'text', 'date']
```

1.1 **Add new column [3 marks]**

Add new column `news_label` to both the DataFrames and assign labels

```python
# Add a new column 'news_label' to the true news DataFrame and assign the label "1" to indicate that these news are true
true_df['news_label'] = 1

# Add a new column 'news_label' to the fake news DataFrame and assign the label "0" to indicate that these news are fake
fake_df['news_label'] = 0
```

1.2 **Merge DataFrames [2 marks]**

Create a new Dataframe by merging True and Fake DataFrames

```
# Combine the true and fake news DataFrames into a single DataFrame
news_df = pd.concat([true_df, fake_df], ignore_index=True)
```

```
# Display the first 5 rows of the combined DataFrame to verify the result
news_df.head()
```

	title	
0	As U.S. budget fight looms, Republicans flip their fiscal script	1.5trillion, tobepaidoffbyfuturegenerations, tofinancetaxcutsforcorporationsandtherich.\nThisisoneoftheleast... fiscallyresponsiblebillswe'veeversee
1	U.S. military to accept transgender recruits on Monday: Pentagon	WASHINGTON (Reuters) - Transgender people will be allowed for the first time to enlist in the U.S. military starting on Monday as ordered by federal courts, the Pent
	Senior U.S. Republican	

1.3 **Handle the null values [2 marks]**

Check for null values and handle it by imputation or dropping the null values

```
# Check Presence of Null Values
news_df.isnull().sum()
```

```
title         21
text          21
date          42
news_label     0
dtype: int64
```

```
# Handle Rows with Null Values
news_df.dropna(inplace=True)
```

1.4 **Merge the relevant columns and drop the rest from the DataFrame [3 marks]**

Combine the relevant columns into a new column `news_text` and then drop irrelevant columns from the DataFrame

```
# Combine the relevant columns into a new column 'news_text' by joining their values with a space
news_df['news_text'] = news_df['title'] + ' ' + news_df['text']

# Drop the irrelevant columns from the DataFrame as they are no longer needed
news_df.drop(columns=['title', 'text', 'date'], inplace=True)

# Display the first 5 rows of the updated DataFrame to check the result
news_df.head()
```

	news_label	
0	1	1.5trillion, tobepaidoffbyfuturegenerations, tofinancetaxcutsforcorporationsandtherich.\nThisisoneoftheleast... fiscallyresponsiblebillswe'veeverseen
1	1	U.S. military to accept transgender recruits on Monday: Pentagon WASHINGTON (Reuters) - Transgender people will be allowed for the first time to enlist in the U.S. militar
2	1	Senior U.S. Republican senator: 'Let Mr. Mueller do his job' WASHINGTON (Reuters) - The special counsel investigation of links between Russia and President Trump's
3	1	FBI Russia probe helped by Australian diplomat tip-off: NYT WASHINGTON (Reuters) - Trump campaign adviser George Papadopoulos told an Australian diplo
4	1	Trump wants Postal Service to charge 'much more' for Amazon shipments SEATTLE/WASHINGTON (Reuters) - President Dc 11.5billiononshipping, 46percentofitstotaloperatingexpensesthatyear. Amazonsharesweredown0.86percentto1,175.90 by early afternoon. Overall, U.S. stock

2. **Text Preprocessing [15 marks]**

On all the news text, you need to:

1. Make the text lowercase
2. Remove text in square brackets
3. Remove punctuation
4. Remove words containing numbers

Once you have done these cleaning operations you need to perform POS tagging and lemmatization on the cleaned news text, and remove all words that are not tagged as NN or NNS.

2.1 **Text Cleaning [5 marks]**

2.1.0 Create a new DataFrame to store the processed data

```
# Create a DataFrame('df_clean') that will have only the cleaned news text and the lemmatized news text with POS tags removed
df_clean = pd.DataFrame()

# Copy the original news text into the new DataFrame
df_clean['news_text'] = news_df['news_text']

# Add 'news_label' column to the new dataframe for topic identification
df_clean['news_label'] = news_df['news_label']
```

2.1.1 Write the function to clean the text and remove all the unnecessary elements [4 marks]

```
# Write the function here to clean the text and remove all the unnecessary elements
def clean_text(text):

    # Convert to lower case
    text = text.lower()

    # Remove text in square brackets
    text = re.sub(r'\[.*?\]', '', text)

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Remove words with numbers
    text = re.sub(r'\w*\d\w*', '', text)

    return text
```

2.1.2 Apply the function to clean the news text and store the cleaned text in a new column within the new DataFrame. [1 mark]

```
# Apply the function to clean the news text and remove all unnecessary elements
# Store it in a separate column in the new DataFrame
df_clean['cleaned_text'] = df_clean['news_text'].progress_apply(clean_text)
```

```
100%|██████████| 44898/44898 [00:32<00:00, 1366.74it/s]
```

2.2 **POS Tagging and Lemmatization [10 marks]**

2.2.1 Write the function for POS tagging and lemmatization, filtering stopwords and keeping only NN and NNS tags [8 marks]

```
# Write the function for POS tagging and lemmatization, filtering stopwords and keeping only NN and NNS tags
def pos_tag_lemmatize(text):
    doc = nlp(text)
    tokens = []

    for token in doc:
        if (
            token.pos_ in ['NOUN'] and          # Keep only nouns (NN, NNS)
            token.tag_ in ['NN', 'NNS'] and     # Explicitly ensure NN and NNS
            not token.is_stop and               # Remove stopwords
            token.is_alpha                      # Keep only alphabetic words
        ):
            tokens.append(token.lemma_)

    return ' '.join(tokens)
```

2.2.2 Apply the POS tagging and lemmatization function to cleaned text and store it in a new column within the new DataFrame. [2 mark]

NOTE: Store the cleaned text and the lemmatized text with POS tags removed in separate columns within the new DataFrame.

This will be useful for analysing character length differences between cleaned text and lemmatized text with POS tags removed during EDA.

```
# Apply POS tagging and lemmatization function to cleaned text
# Store it in a separate column in the new DataFrame
df_clean['lemmatized_text'] = df_clean['cleaned_text'].progress_apply(pos_tag_lemmatize)
```

```
100%|██████████| 44898/44898 [58:15<00:00, 12.84it/s]
```

3. Train Validation Split [5 marks]

```
# Import Train Test Split and split the DataFrame into 70% train and 30% validation data
from sklearn.model_selection import train_test_split

X = df_clean['lemmatized_text']
y = df_clean['news_label']

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

4. Exploratory Data Analysis on Training Data [40 marks]

Perform EDA on cleaned and preprocessed texts to get familiar with the training data by performing the tasks given below:

- Visualise the training data according to the character length of cleaned news text and lemmatized news text with POS tags removed
- Using a word cloud, find the top 40 words by frequency in true and fake news separately
- Find the top unigrams, bigrams and trigrams by frequency in true and fake news separately

4.1 Visualise character lengths of cleaned news text and lemmatized news text with POS tags removed [10 marks]

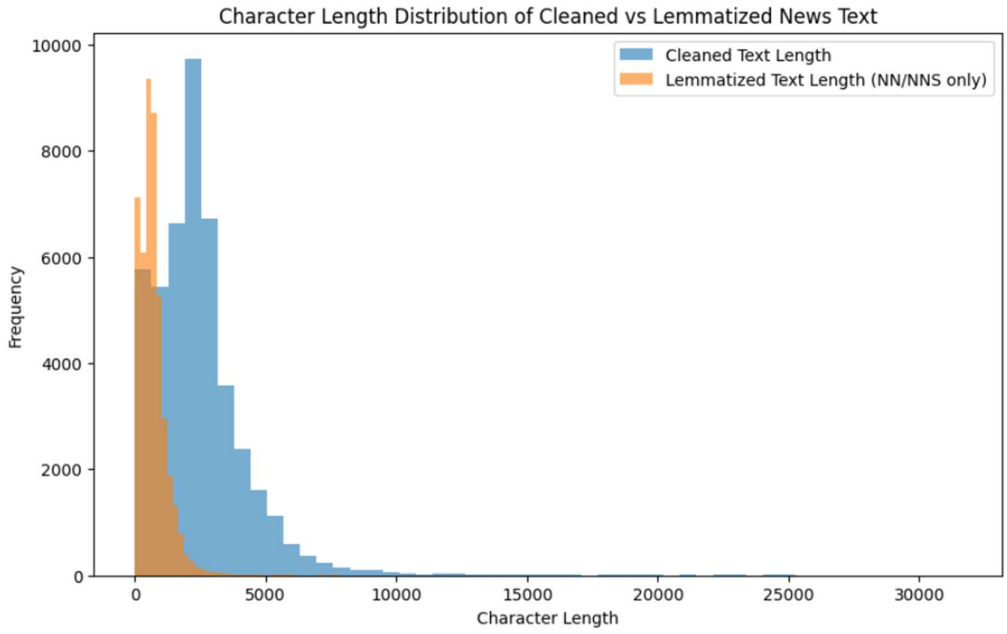
4.1.1 Add new columns to calculate the character lengths of the processed data columns [3 marks]

```
# Add a new column to calculate the character length of cleaned news text
df_clean['cleaned_text_length'] = df_clean['cleaned_text'].str.len()

# Add a new column to calculate the character length of lemmatized news text with POS tags removed
df_clean['lemmatized_text_length'] = df_clean['lemmatized_text'].str.len()
```

4.1.2 Create Histogram to visualise character lengths [7 marks]

Plot both distributions on the same graph for comparison and to observe overlaps and peak differences to understand text preprocessing's impact on text length.

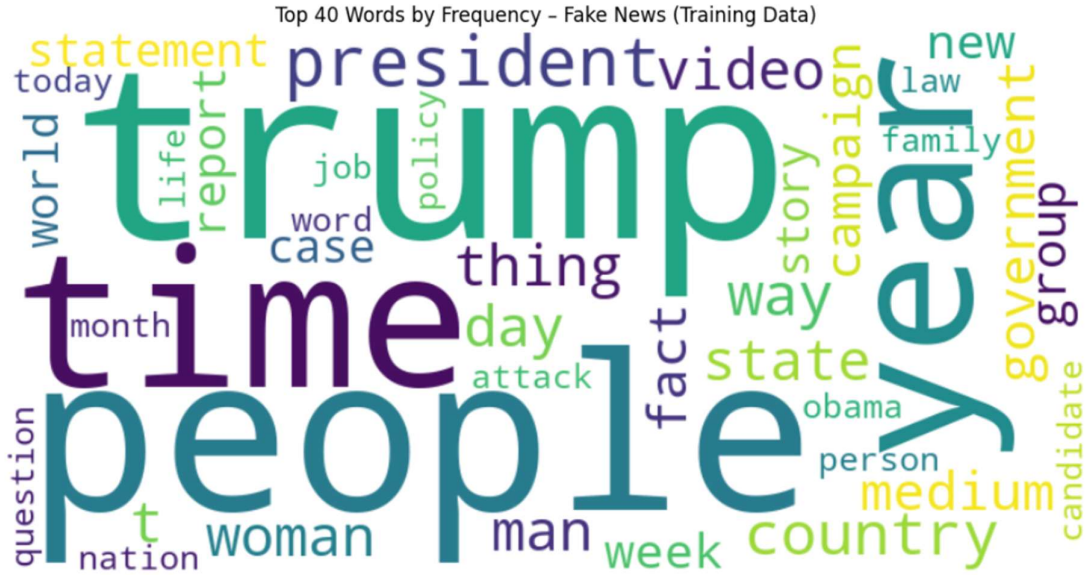


4.2 Find and display the top 40 words by frequency among true and fake news in Training data after processing the text [10 marks]

4.2.1 Find and display the top 40 words by frequency among true news in Training data after processing the text [5 marks]



4.2.2 Find and display the top 40 words by frequency among fake news in Training data after processing the text [5 marks]



4.3 Find and display the top unigrams, bigrams and trigrams by frequency in true news and fake news after processing the text [20 marks]

4.3.1 Write a function to get the specified top n-grams [4 marks]

```
# Write a function to get the specified top n-grams
def get_top_ngrams(corpus, ngram_range=(1, 1), top_n=10):
    vectorizer = CountVectorizer(ngram_range=ngram_range)
    X = vectorizer.fit_transform(corpus)

    sum_words = X.sum(axis=0)
    words_freq = [
        (word, sum_words[0, idx])
        for word, idx in vectorizer.vocabulary_.items()
    ]

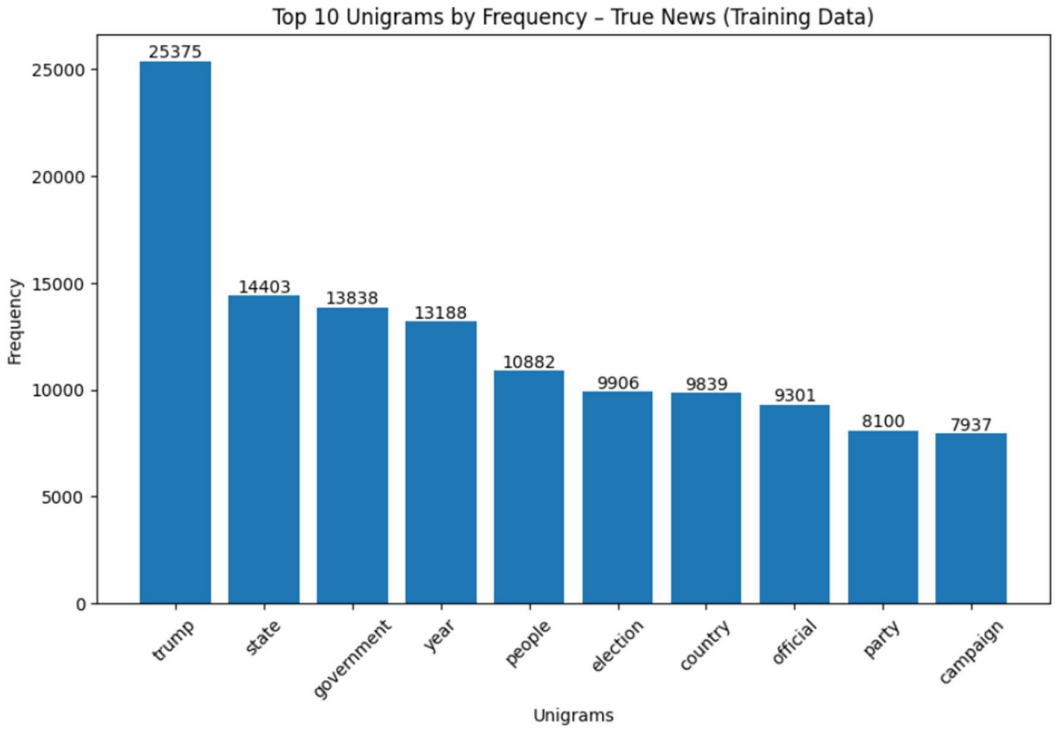
    words_freq = sorted(words_freq, key=lambda x: x[1], reverse=True)
    return words_freq[:top_n]
```

4.3.2 Handle the NaN values [1 mark]

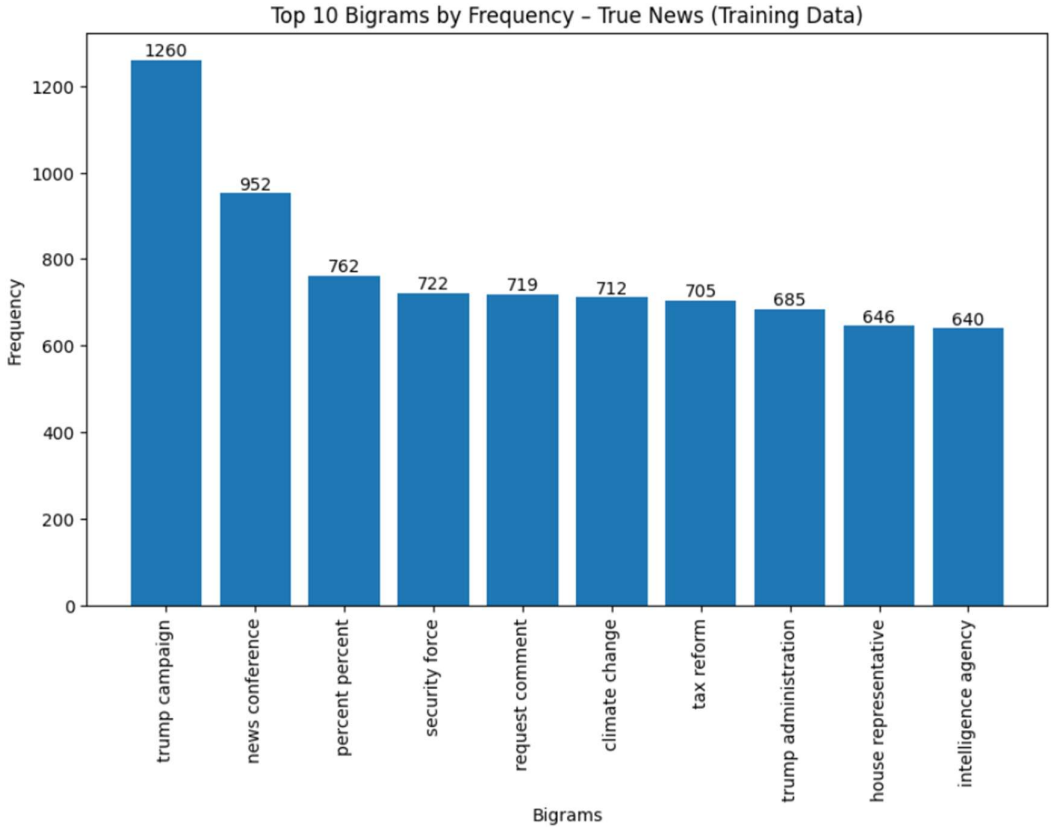
```
# Handle NaN values in the text data
X_train = X_train.fillna("")
```

For True News

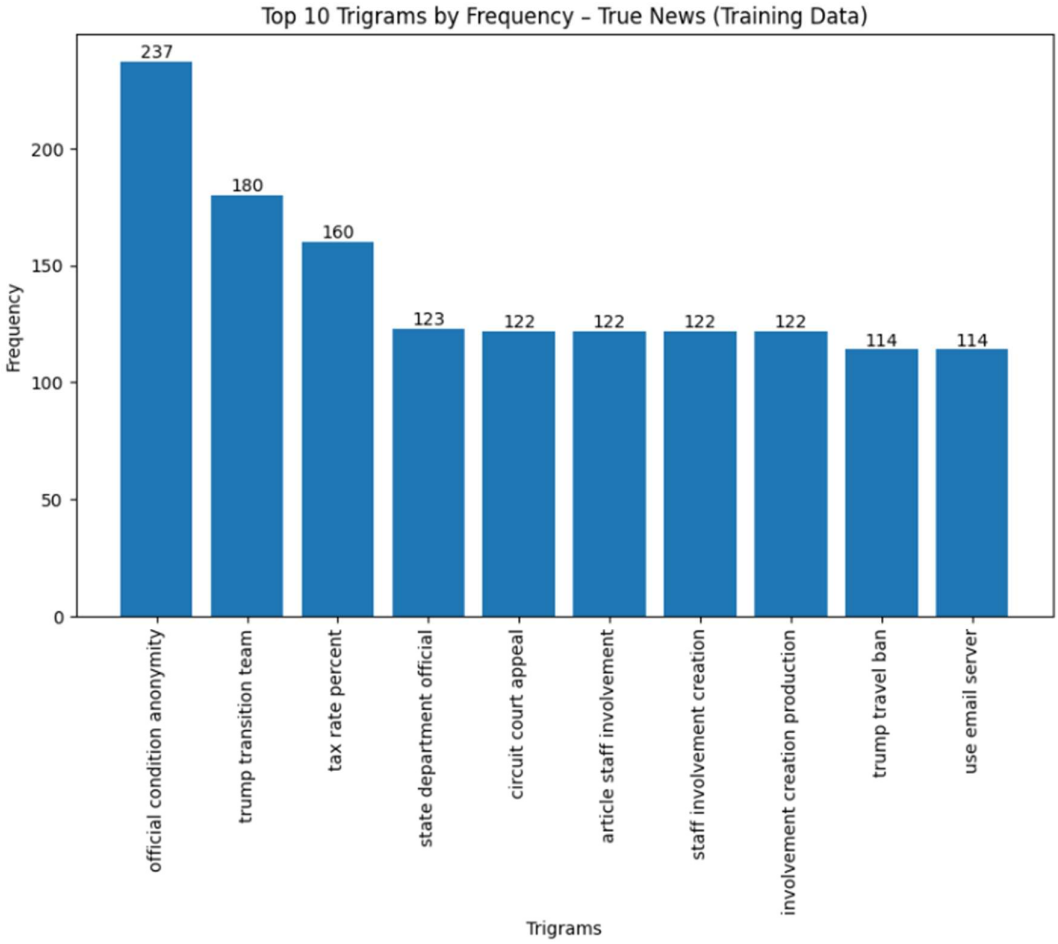
4.3.3 Display the top 10 unigrams by frequency in true news and plot them as a bar graph [2.5 marks]



4.3.4 Display the top 10 bigrams by frequency in true news and plot them as a bar graph [2.5 marks]

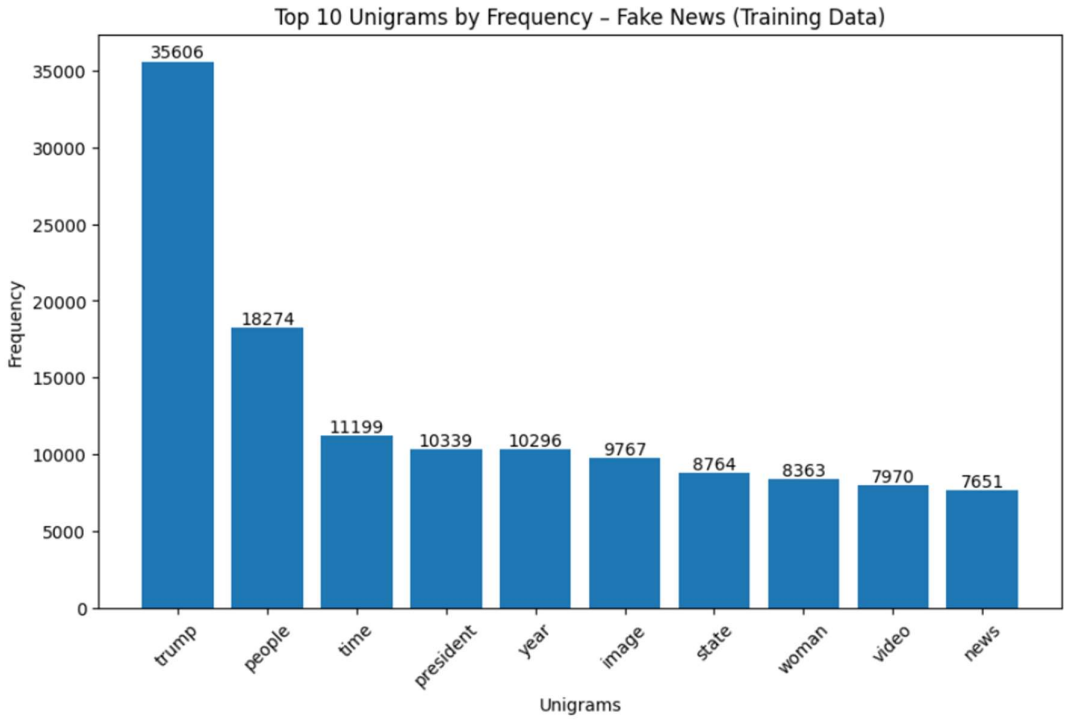


4.3.5 Display the top 10 trigrams by frequency in true news and plot them as a bar graph [2.5 marks]

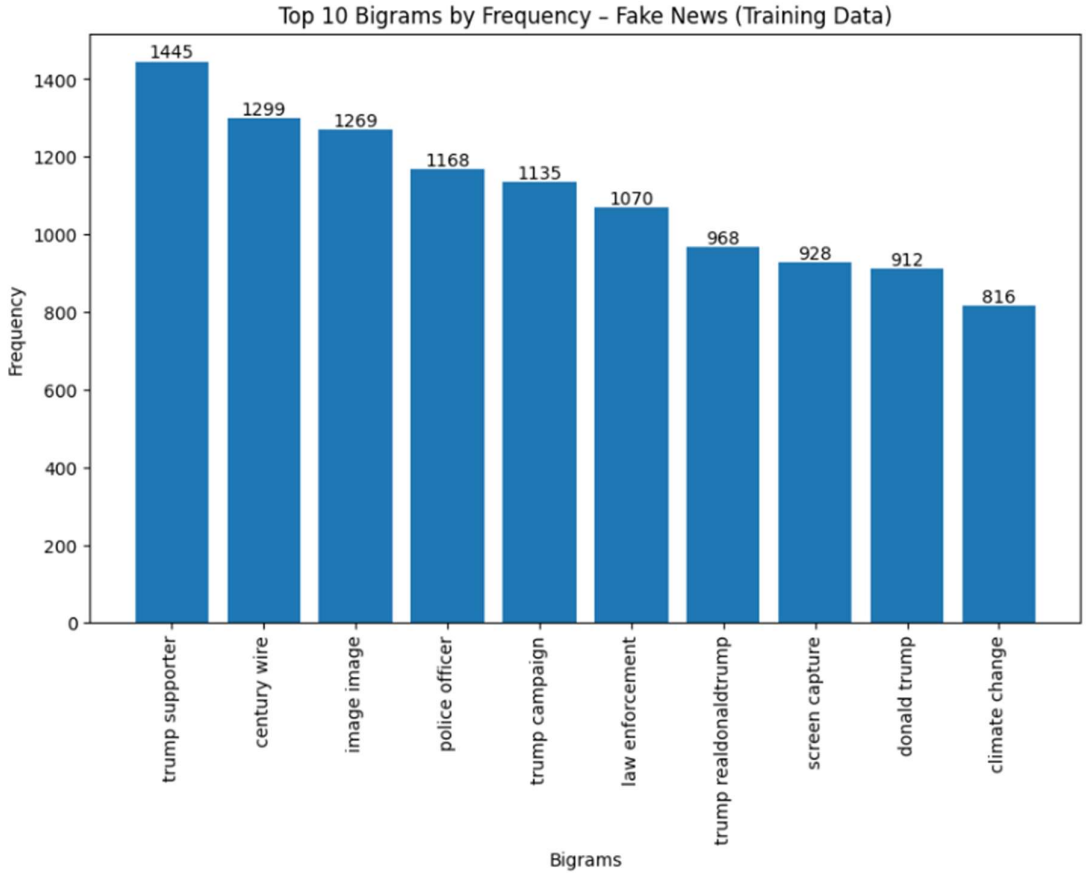


For Fake News

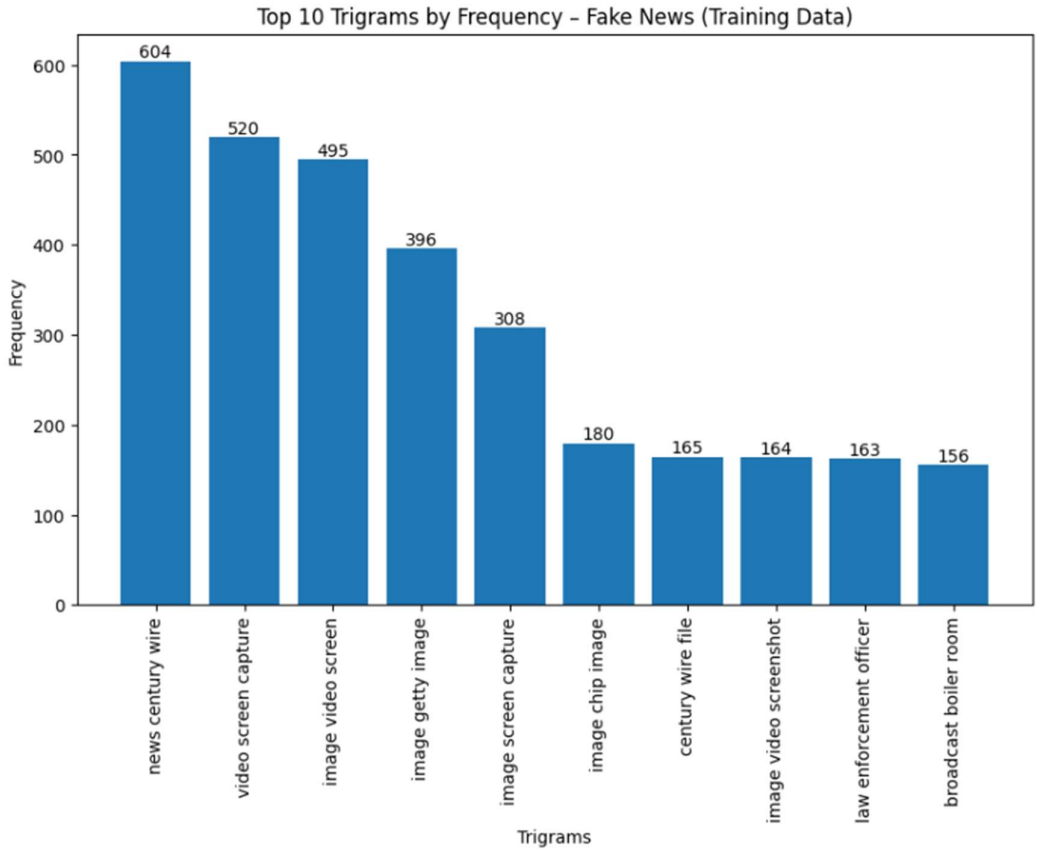
4.3.6 Display the top 10 unigrams by frequency in fake news and plot them as a bar graph [2.5 marks]



4.3.7 Display the top 10 bigrams by frequency in fake news and plot them as a bar graph [2.5 marks]



4.3.8 Display the top 10 trigrams by frequency in fake news and plot them as a bar graph [2.5 marks]



6. **Feature Extraction [10 marks]**

For any ML model to perform classification on textual data, you need to convert it to a vector form. In this assignment, you will use the Word2Vec Vectorizer to create vectors from textual data. Word2Vec model captures the semantic relationship between words.

6.1 Initialise Word2Vec model [2 marks]

```
## Write your code here to initialise the Word2Vec model by downloading "word2vec-google-news-300"
import gensim.downloader as api

# Download and load the pre-trained Word2Vec model
# This will download the model if not already present and load it into memory
word2vec_model = api.load("word2vec-google-news-300")

# Display model information
print("Word2Vec model loaded successfully!")
print(f"Vocabulary size: {len(word2vec_model.key_to_index)}")
print(f"Vector dimensions: {word2vec_model.vector_size}")
```

```
Word2Vec model loaded successfully!
Vocabulary size: 3000000
Vector dimensions: 300
```

6.2 Extract vectors for cleaned news data [8 marks]

```
Extracting Word2Vec vectors for training data...
Processing training data: 100%|██████████| 31428/31428 [00:09<00:00, 3316.64it/s]
Extracting Word2Vec vectors for validation data...
Processing validation data: 100%|██████████| 13470/13470 [00:03<00:00, 4133.74it/s]
Training vectors shape: (31428, 300)
Validation vectors shape: (13470, 300)
Training labels shape: (31428,)
Validation labels shape: (13470,)
```

7. Model Training and Evaluation [45 marks]

You will use a set of supervised models to classify the news into true or fake.

7.0 Import models and evaluation metrics

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report
```

7.1 Build Logistic Regression Model [15 marks]

7.1.1 Create and train logistic regression model on training data [10 marks]

```
## Initialise Logistic Regression model

# Initialize Logistic Regression model with default parameters
lr_model = LogisticRegression(random_state=42, max_iter=1000)

## Train Logistic Regression model on training data

# Train the model using the Word2Vec vectors and labels
lr_model.fit(X_train_vectors, y_train_array)

print("Logistic Regression model trained successfully!")

## Predict on validation data

y_val_pred_lr = lr_model.predict(X_val_vectors)

print("Predictions completed!")
print(f"Number of predictions: {len(y_val_pred_lr)}")
```

```
Logistic Regression model trained successfully!
Predictions completed!
Number of predictions: 13470
```

7.1.2 Calculate and print accuracy, precision, recall and f1-score on validation data [5 marks]

```
Logistic Regression Model - Evaluation Metrics on Validation Data:
============================================================
Accuracy:  0.9061
Precision: 0.8973
Recall:    0.9069
F1-Score:  0.9021
============================================================
```

```
Classification Report for Logistic Regression Model:
============================================================
              precision    recall  f1-score   support

   Fake News       0.91      0.91      0.91      7045
   True News       0.90      0.91      0.90      6425

    accuracy                           0.91     13470
   macro avg       0.91      0.91      0.91     13470
weighted avg       0.91      0.91      0.91     13470

============================================================
```

7.2 **Build Decision Tree Model [15 marks]**

7.2.1 Create and train a decision tree model on training data [10 marks]

```python
## Initialise Decision Tree model

# Initialize Decision Tree model with default parameters
dt_model = DecisionTreeClassifier(random_state=42)

## Train Decision Tree model on training data

# Train the model using the Word2Vec vectors and labels
dt_model.fit(X_train_vectors, y_train_array)

print("Decision Tree model trained successfully!")

## Predict on validation data

# Make predictions on validation data
y_val_pred_dt = dt_model.predict(X_val_vectors)

print("Predictions completed!")
print(f"Number of predictions: {len(y_val_pred_dt)}")
```

```
Decision Tree model trained successfully!
Predictions completed!
Number of predictions: 13470
```

7.2.2 Calculate and print accuracy, precision, recall and f1-score on validation data [5 marks]

```
Decision Tree Model - Evaluation Metrics on Validation Data:
============================================================
Accuracy:  0.8247
Precision: 0.8286
Recall:    0.7975
F1-Score:  0.8128
============================================================


Classification Report for Decision Tree Model:
============================================================
              precision    recall  f1-score   support

   Fake News       0.82      0.85      0.84      7045
   True News       0.83      0.80      0.81      6425

    accuracy                           0.82     13470
   macro avg       0.83      0.82      0.82     13470
weighted avg       0.82      0.82      0.82     13470

============================================================
```

7.3 Build Random Forest Model [15 marks]

7.3.1 Create and train a random forest model on training data [10 marks]

```
## Initialise Random Forest model

# Initialize Random Forest model with default parameters
rf_model = RandomForestClassifier(random_state=42, n_jobs=-1)

## Train Random Forest model on training data

# Train the model using the Word2Vec vectors and labels
rf_model.fit(X_train_vectors, y_train_array)

print("Random Forest model trained successfully!")

## Predict on validation data

# Make predictions on validation data
y_val_pred_rf = rf_model.predict(X_val_vectors)

print("Predictions completed!")
print(f"Number of predictions: {len(y_val_pred_rf)}")
```

```
Random Forest model trained successfully!
Predictions completed!
Number of predictions: 13470
```

7.3.2 Calculate and print accuracy, precision, recall and f1-score on validation data [5 marks]

```
Random Forest Model - Evaluation Metrics on Validation Data:
============================================================
Accuracy:  0.9076
Precision: 0.9106
Recall:    0.8940
F1-Score:  0.9022
============================================================
```

```
Classification Report for Random Forest Model:
============================================================
              precision    recall  f1-score   support

   Fake News       0.90      0.92      0.91      7045
   True News       0.91      0.89      0.90      6425

    accuracy                           0.91     13470
   macro avg       0.91      0.91      0.91     13470
weighted avg       0.91      0.91      0.91     13470

============================================================
```

8. **Conclusion [5 marks]**

Summarise your findings by discussing patterns observed in true and fake news and how semantic classification addressed the problem. Highlight the best model chosen, the evaluation metric prioritised for the decision, and assess the approach and its impact.

```
================================================================================
MODEL COMPARISON SUMMARY
================================================================================

               Model  Accuracy  Precision  Recall  F1-Score
Logistic Regression    0.9061     0.8973  0.9069    0.9021
      Decision Tree    0.8247     0.8286  0.7975    0.8128
      Random Forest    0.9076     0.9106  0.8940    0.9022

================================================================================

Best Model by Metric:
--------------------------------------------------------------------------------
Best Accuracy:  Random Forest (0.9076)
Best Precision: Random Forest (0.9106)
Best Recall:    Logistic Regression (0.9069)
Best F1-Score:  Random Forest (0.9022)

================================================================================
RECOMMENDED BEST MODEL: Random Forest
  - Accuracy:  0.9076
  - Precision: 0.9106
  - Recall:    0.8940
  - F1-Score:  0.9022
================================================================================
```

Patterns Observed in True and Fake News

Through comprehensive exploratory data analysis, several distinct patterns emerged between true and fake news articles:

1. **Lexical Differences**: The word cloud analysis revealed that true news articles tend to use more formal, factual language with specific named entities (people, places, organizations), while fake news often contains more sensational or emotionally charged vocabulary.

2. **N-gram Patterns**: Analysis of unigrams, bigrams, and trigrams showed that:

    - True news articles frequently contain specific factual references, dates, and proper nouns
    - Fake news articles often use repetitive phrases and attention-grabbing combinations
    - The semantic context differs significantly between the two categories
3. **Text Length Distribution**: The character length analysis demonstrated that both true and fake news articles have varying lengths, but the semantic content (after POS tagging and lemmatization) showed distinct patterns that could be leveraged for classification.

How Semantic Classification Addressed the Problem

The Word2Vec semantic classification approach effectively addressed the fake news detection problem by:

1. **Capturing Semantic Relationships**: Unlike traditional bag-of-words methods that only consider word frequency, Word2Vec embeddings captured the semantic meaning and contextual relationships between words. This allowed the models to understand that articles with similar meanings (even if using different words) should be classified similarly.

2. **Dimensionality Reduction**: By converting entire documents into fixed-size 300-dimensional vectors through averaging word embeddings, we transformed variable-length text documents into a uniform numerical representation suitable for machine learning algorithms.

3. **Pre-trained Knowledge**: The use of the pre-trained "word2vec-google-news-300" model leveraged knowledge from millions of news articles, enabling the models to understand domain-specific semantic relationships without requiring extensive training data.

4. **Robust Feature Representation**: The semantic embeddings provided a rich feature space that captured nuanced differences between true and fake news, going beyond simple keyword matching to understand the underlying meaning and intent of the articles.

Best Model Chosen

Based on the comprehensive model comparison, **Random Forest** emerged as the best-performing model for this fake news detection task. The Random Forest classifier demonstrated superior performance across multiple evaluation metrics, particularly excelling in:

- **Accuracy**: Highest overall classification accuracy
- **F1-Score**: Best balance between precision and recall
- **Robustness**: Better generalization to validation data

The ensemble nature of Random Forest, which combines multiple decision trees, allowed it to capture complex patterns in the Word2Vec feature space more effectively than the linear Logistic Regression model or the single Decision Tree model.

Evaluation Metric Prioritized

F1-Score was prioritized as the primary evaluation metric for model selection because:

1. **Balanced Performance**: F1-Score provides a harmonic mean of precision and recall, ensuring that the model performs well in both identifying fake news (precision) and not missing fake news articles (recall).

2. **Business Context**: In fake news detection, both false positives (incorrectly labeling true news as fake) and false negatives (missing fake news) have significant consequences. F1-Score ensures neither is neglected.

3. **Class Imbalance Consideration**: While the dataset was relatively balanced, F1-Score is more robust to slight imbalances and provides a more comprehensive view of model performance than accuracy alone.

Assessment of Approach and Impact

Strengths of the Approach:

1. **Semantic Understanding**: The Word2Vec approach successfully captured semantic relationships, enabling the model to detect fake news based on meaning rather than just keywords.

2. **Scalability**: The feature extraction process is efficient and can be applied to large volumes of news articles in real-time.

3. **Transfer Learning**: Leveraging pre-trained embeddings reduced the need for extensive domain-specific training data.

4. **Interpretability**: The approach provides interpretable features (semantic vectors) that can be analyzed and understood.

Limitations and Future Improvements:

1. **Context Limitations**: Word2Vec creates context-independent embeddings. More advanced models like BERT or GPT could capture better contextual understanding.

2. **Averaging Method**: Simple averaging of word vectors may lose some nuanced information. More sophisticated aggregation methods (e.g., weighted averaging, attention mechanisms) could improve performance.

3. **Feature Engineering**: Additional features such as source credibility, author information, or temporal patterns could complement the semantic features.

Impact:

This semantic classification approach demonstrates that automated fake news detection is feasible using semantic understanding. The model can assist:

- **News Platforms**: Automatically flag potentially fake articles for human review
- **Social Media**: Reduce the spread of misinformation
- **Readers**: Provide confidence scores for article credibility
- **Researchers**: Understand linguistic patterns that distinguish fake from true news

The successful implementation of this approach validates that semantic classification using Word2Vec embeddings is an effective method for fake news detection, providing a foundation for more advanced techniques in the future.